

MECH 498 - Honours Thesis

Fall 2025



**University
of Victoria**

Final Report:

Bridging the Simulation-to-Reality Gap:

**A Comparative Study of Physics-Based Simulation Platforms
in Robotics**

Student: Liesa Sileshi (V01016713)

Supervisor: Dr Homayoun Najjaran

Abstract

Selecting the right robotics simulation platform is critical for successful robotics research and development. Each platform implements physics engines differently, leading to variations in simulation behaviour and how well results translate to physical hardware, referred to as the simulation-to-reality gap. Understanding these deviations is essential because they directly affect the reliability of simulation results and the likelihood that control strategies developed in simulation will succeed on real robots.

This project presents a quantitative comparison of two widely used simulation platforms, Gazebo (ODE) and Unity (PhysX), using identical industrial robot models (UFactory UF850 and xArm6) and standardized test scenarios. The evaluation focuses on joint-space trajectory tracking and friction-driven sliding on an inclined plane to assess physics accuracy, contact interactions, and computational behaviour under controlled conditions. NVIDIA Isaac Sim is also discussed as a candidate for future work to extend this comparison.

The findings show that Unity can achieve accurate trajectory tracking with relatively low effort needed for controller tuning, while Gazebo's performance is more sensitive to controller configuration and robot model parameters. For friction and sliding, both simulators demonstrate the expected trends but differ in how they handle stopping behaviour and longer durations of motion, highlighting the need to validate simulation performance when precise distance and duration measurements are required. By identifying where each platform follows or deviates from theoretical behaviour, and where modeling choices introduce larger errors, this study helps researchers interpret simulation results more effectively, emphasizing areas where additional calibration is required. It supports more informed decisions about how much confidence to place in simulated outcomes before transferring them to real hardware and contributes to ongoing efforts to better understand and bridge the simulation-to-reality gap.

Table of Contents

Abstract	2
List of Tables	5
List of Figures	6
1. Introduction	7
2. Objective	7
3. Background and Literature Review	7
3.1 Overview of Physics-Based Simulation	7
3.2 Role of Physics-Based Simulation in Robotics	8
3.3 Differences Between Platforms (Gazebo, Unity, NVIDIA Isaac Sim)	9
3.3.1 Gazebo	9
3.3.2 Unity	10
3.3.3 NVIDIA Isaac Sim	10
3.4 Challenges in Physics Simulation: Simulation-to-Reality Gap	11
3.4.1 Contact Modeling and Solver Limitations	11
3.4.2 Other Sources of Discrepancy	12
3.5 Motivation	13
4. Methodology	13
4.1 Hardware/Software Requirements	13
4.2 Robot and Environment Setup	14
4.3 Test scenarios	14
4.3.1 Test 1: Joint-Space Trajectory Tracking	15
4.3.1.1 Experimental Setup	15
4.3.1.2 Implementation	16
4.3.1.3 Data Collection	17
4.3.1.4 Theoretical Comparison	17
4.3.2 Test 2: Friction and Sliding on an Inclined Plane	17
4.3.2.1 Experimental Setup	18
4.3.2.2 Implementation	18
4.3.2.3 Data Collection	19
4.3.2.4 Theoretical Comparison	19
4.3 Success Criteria for Test Implementation	20
5. Results	20
5.1 Test 1: Joint-Space Trajectory Tracking	20
5.1.1 UF850 Trajectory Tracking	21
5.1.2 xArm6 Trajectory Tracking	24
5.1.3 Summary of Trajectory Tracking Performance	28
5.2 Test 2: Friction and Sliding on an Inclined Plane	29
5.2.1 Overview of Results	29
5.2.2 Position vs Time Analysis	30
5.2.3 Speed vs Time Analysis	30
5.2.4 Comparison of Performance Metrics	31

5.2.5 Error Analysis	31
6. Discussion	32
6.1 Test 1: Joint-Space Trajectory Tracking	32
6.1.1 Tracking Accuracy: Gazebo vs Unity	32
6.1.2 Effect of Trajectory Complexity	33
6.1.3 Implications for Choosing a Simulator	33
6.2 Test 2: Friction and Sliding on an Inclined Plane	33
6.2.1 Overview of Results	33
6.2.2 Implications for Choosing a Simulator	34
7. Future Work	35
7.1 Refining Experiment Design	35
7.2 Further Platform Comparison	36
7.3 Real World Validation	36
8. References	37

List of Tables

Table 1. UF850 Working Range from UFactory User Manual, 2023 [16].	16
Table 2. xArm6 Working Range from UFactory User Manual, 2023 [17].	16
Table 3. UF850 and xArm6 Motion Parameters from UFactory User Manual, 2023 [16][17].	16
Table 4. Summary of Trajectory Tracking Results for all Trials	28
Table 5. Max Speed Data from Each Platform and Friction Condition	29
Table 6. Distance Data from Each Platform and Friction Condition	29
Table 7. Duration Data from Each Platform and Friction Condition	29
Table 8. Summary of Error Analysis for Each Friction Condition and Platform	31

List of Figures

Figure 1. Unity Environment for Trajectory Tracking Experiment	15
Figure 2. Unity Simulation of Low (green), Medium (blue), and High (red) Friction Conditions	17
Figure 3. UF850 Basic Trajectory Tracking Data	21
Figure 4. UF850 Basic Trajectory Tracking Error	22
Figure 5. UF850 Complex Trajectory Tracking Data	23
Figure 6. UF850 Complex Trajectory Tracking Error	24
Figure 7. xArm6 Basic Trajectory Tracking Data	25
Figure 8. xArm6 Basic Trajectory Tracking Error	26
Figure 9. xArm6 Complex Trajectory Tracking Data	27
Figure 10. xArm6 Complex Trajectory Tracking Error	28
Figure 11. Position vs Time Comparison for Each Platform and Friction Condition	30
Figure 12. Speed vs Time Comparison for Each Platform and Friction Condition	30
Figure 13. Comparison of Performance Metrics for Each Platform and Friction Condition	31
Figure 14. Percent Error Comparison for Each Platform and Friction Condition	31

1. Introduction

Simulation is the imitation of a real-world process or system created in a virtual environment [1]. It has become a fundamental tool used in design, development, and deployment for robotics research and industrial automation. Robots are no longer only tested on hardware; control logic, trajectory planning, and safety checks can all be done virtually prior to testing on hardware. Simulations that can accurately reproduce physical behaviour, contact dynamics, and actuator responses allow engineers to test complex scenarios without risk.

Not all simulation platforms are created equal. Audonnet et al. demonstrate that even though simulators exhibit similar behaviours, underlying differences in physics solvers, collision handling, and joint articulation are present among all platforms [2]. These small differences in simulation responses can propagate into larger errors when transitioning to implementation on real machines. Recognizing this challenge in robotics simulation, Kargar's [3] 2024 survey of realistic robot simulations emphasizes the importance of comparing fidelity, scalability, and computational efficiency when selecting the appropriate platform.

In this study, three widely-used simulation environments: Gazebo, Unity, and NVIDIA Isaac Sim are explored, while the experimental testing focuses specifically on Gazebo and Unity.

2. Objective

This project aims to provide quantitative data to compare simulation platforms, Gazebo and Unity, using identical robot models (UFactory UF850 and xArm6) and standardized test scenarios. The goal is to help researchers make informed platform selection decisions based on performance metrics, and to establish a repeatable methodology for evaluating physics accuracy, contact behaviour, and controller performance that can be extended to additional platforms such as NVIDIA Isaac Sim in future work.

Background and Literature Review

3.1 Overview of Physics-Based Simulation

Physics-based simulations are computational models that can approximate the motion and interaction of bodies governed by the laws of dynamics, including rigid-body kinematics, forces, torques, and contact phenomena like collisions and friction [4]. In robotics, these types of simulators usually represent a robot as a system of rigid links connected by joints, with actuators that apply forces, and sensors that provide observations of the virtual environment. By integrating the equations of motion over discrete time steps, the simulator produces an estimation for the trajectories of joint angles, velocities, and forces, demonstrating how it would appear in the real world [4][5].

3.2 Role of Physics-Based Simulation in Robotics

Physics-based simulation plays an increasingly important role in modern robotics. It is a tool that allows engineers to predict and analyze how robots behave in the real-world without the need for extensive physical testing, which reduces both the risk and cost throughout the early stages of development. Liu and Negrut [6] explain how these virtual environments provide a safe and accelerated space for developing and validating mechanical designs, sensors, and control algorithms. Similarly, Thaker [7] refers to physics-based simulations as a fast, safe, and cost effective environment for virtual prototyping of robot dynamics and environment interactions.

Liu and Negrut [6] highlight how simulation has become an ideal set up for developing control strategies and testing robot performance under different conditions. Since simulation can recreate identical test conditions, it allows for easily repeated experiments that are otherwise difficult or even impossible to achieve physically [6]. They can reproduce the exact same initial states and disturbances, making it an ideal tool for comparing algorithms or performing sensitivity studies. This repeatability allows researchers to isolate and understand the effects of individual parameters, explore edge cases, and evaluate failure modes without risking equipment damage or safety [7].

Outside of design and testing, simulation plays a crucial role in training and validation. For autonomous and AI-integrated robotics applications, simulation platforms can be used to generate large volumes of labelled data that would take long periods of time to collect on physical systems. This is especially valuable for machine learning applications where robots have to learn through trial and error. Through simulations, experiments can be performed safely, in high volumes, and at a low cost [6][7]. With the computing power available today, simulations can approximate physical laws with high speed and resolution, making them a staple tool for most modern robotics research and development.

As simulation continues to become a more central part of robotics development, various physics-based engines have emerged to support different applications and needs. These engines can range from more general purpose engines like ODE and Bullet, to more recent robotics focused engines including DART, MuJoCo, RaiSim, and Drake. Different engines vary in their mathematical formulations, coordinate systems, and approach to solving contact constraints [4]. This leads to trade-offs between computational speed, numerical stability, and physical accuracy, making each engine better suited to specific robotics applications. This reinforces the importance of choosing the correct simulator based on the goals of a project.

3.3 Differences Between Platforms (Gazebo, Unity, NVIDIA Isaac Sim)

Each of the three simulation platforms explored in this report have a unique approach to physics and user interaction. All platforms are designed to replicate the behaviour of real world robotic systems, however, they each go about it with different priorities and assumptions. For example, some platforms focus more on accurate physics modelling and integration with robotics middleware like ROS2, while others prioritize advanced graphics, real-time rendering, or compatibility with machine learning frameworks [8]. These design choices directly affect how robots behave in simulation and the types of research or industrial applications each platform supports. As noted by Liu and Negrut [4], generic physics engines often vary in how they balance realism and speed, with no single solution performing best across all tasks. This variation makes it important to understand the characteristics and limitations of each platform before interpreting simulation results or comparing performance across environments.

3.3.1 Gazebo

Gazebo is an open-source simulation platform that is integrated with the ROS (Robot Operating System) ecosystem [9]. Gazebo supports multiple configurable physics engines that users can choose from based on their specific simulation needs. Gazebo currently supports four different physics engines including Dynamic Animation and Robotics Toolkit (DART) , Open Dynamics Engine (ODE), Bullet, and Simbody. Each engine uses different mathematical and computational approaches to simulation, for example, ODE and Bullet use coordinates optimized for performance in complex scenes with independent objects [9]. In contrast, DART and Simbody use generalized coordinates that are better suited to robot manipulators [9]. Gazebo's support for URDF (Universal Robot Description Format) and close integration with ROS simplifies the process of testing control software in simulation before deploying to physical systems.

3.3.2 Unity

Unity is a game engine that is becoming more commonly used for robotics simulation and machine learning research [10]. Unlike Gazebo, it is not specifically designed for robotics; instead, it prioritizes real-time 3D graphics rendering for game development. Unity's 3D physics simulation is provided by the built-in NVIDIA PhysX engine, which is optimized for visual plausibility and performance at high frame rates rather than strict physical accuracy [10]. This design choice creates a trade-off where Unity excels at creating visual simulations with high fidelity graphics, while sacrificing some precision in the physics calculations for computational speed [11].

Unity integrates with ROS through specialized middleware tools. For example, using the ROS-TCP-Connector package, developers can synchronize simulation data with physical robots for digital twin applications [11]. Additionally, the URDF importer tool allows robot description files to be directly loaded into Unity. However, accessing and modifying physics parameters requires navigating the editor interface or writing C# scripts, which can be less transparent than Gazebo's configuration approach. Careful parameter tuning is needed to achieve sufficient physics fidelity for research applications [10][11].

3.3.3 NVIDIA Isaac Sim

NVIDIA Isaac Sim is a physics simulation platform built specifically for robotics research and industrial applications. It uses NVIDIA PhysX as its physics engine with photorealistic rendering and is designed to support large-scale simulation and AI training [10]. It has comprehensive robotics tooling including pre-built robot models, URDF support, advanced sensor simulation, and ROS integration [12].

A key feature is its GPU-accelerated capability, which allows multiple robot simulations to run in parallel on a single GPU, allowing rapid generation of large-scale synthetic training data [12][13]. Zhou et al. [13] explore this capability, using Isaac Sim for rapid prototyping and evaluation of AI controllers through eight different manipulation experiments. Their work demonstrates how the platform can accelerate the development of learning-based control policies by generating large amounts of simulated robot experience in a fraction of the time required by CPU-based simulators [13].

Isaac Sim provides high-fidelity physics simulations; however, these capabilities require substantial computational resources with specialised hardware [10]. Additionally, the simulation-to-reality gap remains a challenge when deploying control policies trained in simulation to physical robots.

3.4 Challenges in Physics Simulation: Simulation-to-Reality Gap

Although physics-based simulators aim to model the same physical laws that govern real robotic systems, they each make necessary approximations and simplifications that can cause simulated behaviour to differ from what happens in reality. Thaker [7] explains that while simulators incorporate factors such as rigid-body dynamics, contact, and friction to create realistic environments, they involve complex calculations and modeling choices that prioritize computational efficiency over accuracy. As a result, even the most carefully developed simulations can deviate from physical reality. This is especially noticeable in scenarios that involve complex contact interactions or highly dynamic motion [7].

3.4.1 Contact Modeling and Solver Limitations

One of the main sources of deviation between simulation and reality come from how contact and friction are modeled and solved. Le Ledec et al. demonstrate how various physics engines implement fundamentally different contact models and numerical solvers, even though all are based on the same physical principles [4]. For example, simulators approximate contact constraints using different formulations such as linear complementarity problems (LCP), nonlinear complementarity problems (NCP), or cone complementarity problems (CCP). Each formulation involves different relaxations and numerical approximations, and have a direct impact on the simulator behaviour. Specific solver algorithms, convergence criteria, and friction approximations used by each engine can significantly affect contact forces, sticking and slipping behaviours, and impact responses [4]. Their quantitative benchmarks indicate that some commonly used approximations in robotics simulators can “severely widen the reality gap” and impact applications that require precise contact interactions [4]. While it can be treated as a secondary consideration in robotics research, contact modeling is an important factor that determines whether simulated control policies will transfer successfully to real hardware.

3.4.2 Other Sources of Discrepancy

Beyond contact modeling, the “simulation-to-reality gap” includes multiple other sources of deviation between simulated and physical systems. Muratore et al. [14] describe the reality gap as an inevitable consequence of working with model-based simulators, which are unable to capture the full complexity of the real world. Even with a physics engine that can reasonably model rigid-body dynamics, inaccuracies in friction coefficients, material compliance, actuator dynamics, and sensor noise could all contribute to simulated policies failing when implemented on real robots [14]. Aljalbout et al. [15] provide a comprehensive classification of the reality gap, categorizing it into four components: the dynamics gap (differences in mass, inertia, friction, contact, and numerical integration), the perception gap (differences in sensor models, lighting, textures, and environment representation), the actuation gap (unmodeled actuator nonlinearities, delays, and power electronics), and the system design gap (differences in communication latency, safety mechanisms, and formulations).

Current approaches to managing the reality gap combine multiple strategies. Domain randomization varies physical and visual parameters to develop robust policies. System identification matches simulator parameters to real hardware through tuning and iterative refinement. Residual learning augments imperfect simulators with neural networks that correct for unmodelled dynamics [15]. Even with these advanced strategies, the reality gap cannot be completely eliminated. Instead, the gap must be managed and characterized so that simulation results can be interpreted properly and reliably transferred to physical robots.

3.5 Motivation

The motivation for this project is to provide quantitative, physics-based evidence on how Gazebo and Unity differ. It explores how each platform handles contact dynamics, joint constraints, and control accuracy when simulating identical industrial robot systems. Understanding these simulator differences is important because they directly affect how trustworthy the results are and how well results will translate to real hardware. By establishing reproducible test scenes, identical initial conditions, and quantitative metrics across platforms, this project will help researchers make informed decisions about platform selection. It will provide the information needed to interpret simulation results appropriately and make realistic predictions about the simulation-to-reality transfer.

4. Methodology

This project compares two physics-based simulation platforms: Gazebo (ODE), Unity (PhysX) which were the focus of this study. NVIDIA Isaac Sim was not implemented in this stage of the project but was recommended as future work for comparison between the three platforms.

Comparison is done using identical industrial robot models (UFactory UF850 and xArm6), and a set of representative test scenarios. The goal is to quantify how each platforms' specific physics solvers, contact models, and implementations impact simulation fidelity and usability for robotics applications.

4.1 Hardware/Software Requirements

- Windows 11 host with WSL2
- Ubuntu 24.04 LTS (Noble) running in WSL2
- ROS 2 Jazzy (installed on Ubuntu 24.04)
- Gazebo Harmonic (gz sim)
- MoveIt2 for ROS 2 Jazzy
- Python 3.12.3
- Python packages: pandas, matplotlib, numpy, os
- Unity 2022.3.62f2

4.2 Robot and Environment Setup

Two industrial robots are used as primary test systems:

- UFactory UF850
- UFactory xArm6

For each robot, the same URDF files are used across all simulators with minimal modifications for compatibility (mesh paths, inertial parameters, joint limits). Initial poses, object placements, and physical parameters (masses, friction coefficients) are kept identical to ensure observed differences reflect simulator behaviour.

The simulated environment includes:

- Fixed base for each robot
- A flat table surface positioned in front of the robot
- Simple geometric objects (ex: cubes, inclined ramp) for contact tasks

4.3 Test scenarios

The following test scenarios are designed to provide information about how each simulator handles basic robot motion, contact, and fundamental physics behaviour.

4.3.1 Test 1: Joint-Space Trajectory Tracking

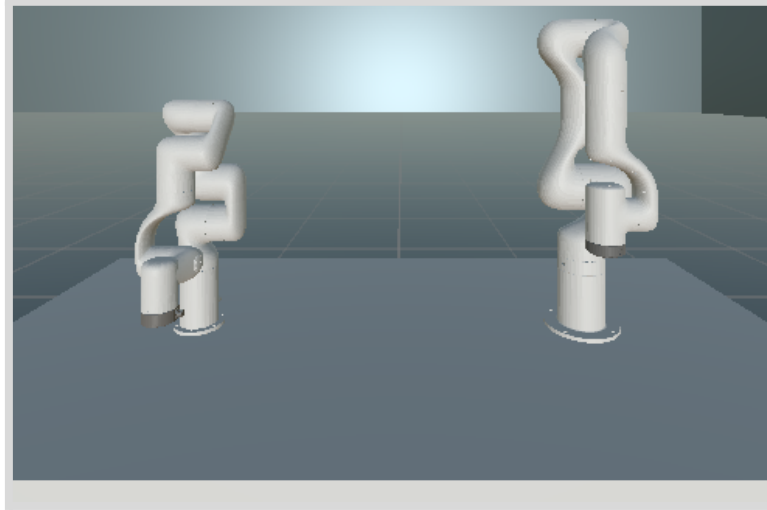


Figure 1. Unity Environment for Trajectory Tracking Experiment

This test compares how accurately each simulator can follow an identical joint-space trajectory for each of the two industrial robot models. By directing the xArm6 and UF850 to execute predefined joint motions, this test is able to isolate how each physics engine handles joint dynamics, interpolation between defined points, and response of articulated bodies driven by position commands. The resulting joint position information provides insight on the trajectory tracking performance of each simulator under controlled conditions.

4.3.1.1 Experimental Setup

For this test, each industrial manipulator is mounted onto a flat table with no obstacles in the workspace. The purpose of the simplistic environment is so that any differences in motion are mainly a reflection of the tracking behaviour rather than the result of contact dynamics or other interference. The same URDF is used for both robots across all platforms. The manufacturer-specified joint ranges and motion limits for the UF850 and xArm6 are summarized in Table 1, Table 2, and Table 3 below. These tables list the allowable rotation range for each joint and maximum joint speed, acceleration, and jerk, which were considered when designing the trajectories to remain within those bounds.

Table 1. UF850 Working Range from UFactory User Manual, 2023 [16].

	Robotic Arm	UFACTORY 850
Maximum		180°/s
Working Range	Joint 1	±360°
	Joint 2	-132° ~ 132°
	Joint 3	-242° ~ 3.5°
	Joint 4	±360°
	Joint 5	-124° ~ 124°
	Joint 6	±360°

Table 2. xArm6 Working Range from UFactory User Manual, 2023 [17].

	Robotic Arm	xArm 6
Maximum		180°/s
Working Range	Joint 1	±360°
	Joint 2	-118° ~ 120°
	Joint 3	-225° ~ 11°
	Joint 4	±360°
	Joint 5	-97° ~ 180°
	Joint 6	±360°
	Joint 7	None

Table 3. UF850 and xArm6 Motion Parameters from UFactory User Manual, 2023 [16][17].

	TCP Motion	Joint Motion
Speed	0 ~ 1000mm /s	0 ~ 180°/s
Acceleration	0 ~ 50000mm /s ²	0 ~ 1145°/s ²
Jerk	0 ~ 100000mm /s ³	0 ~ 28647°/s ³

All simulations used the same gravity setting: -9.81 m/s^2 in the global vertical direction.

4.3.1.2 Implementation

In Gazebo, both the UF850 and xArm6 were spawned using ROS 2 launch files, and joint motion is driven by a position controller node that publishes joint commands at a fixed rate. The node reads joint trajectories and interpolates between the defined points and their time stamps. The command joint positions were then published to the `/joint_trajectory_controller` while Gazebo simulates the motion using the ODE physics engine. For Unity, the same URDF models were imported using Unity URDF Importer, and controlled using C# scripts attached to robots. The scripts contained the same trajectory information and determined the

commanded angle for every fixed timestep. The angles are then commanded to the corresponding *ArticulationBody* joint drives.

4.3.1.3 Data Collection

Throughout each trajectory tracking test, the simulators recorded the commanded joint positions and the actual simulator joint positions over time. In Gazebo, these values were published on the ROS 2 joint state topic and logged to a CSV file. In Unity, the C# control scripts recorded the same fields at every time-step, into a CSV file. The data was then collected from the CSV files and the tracking error was computed using python as the difference between the desired and actual joint angles at each timestep.

4.3.1.4 Theoretical Comparison

The theoretical reference for these tests were the desired joint positions and times. The tracking error was used to quantify the difference between the theoretical trajectory and the simulated trajectory. Comparing the tracking errors allows for comparison of each simulator based on how accurate it reproduces the desired joint motion under the same conditions.

4.3.2 Test 2: Friction and Sliding on an Inclined Plane

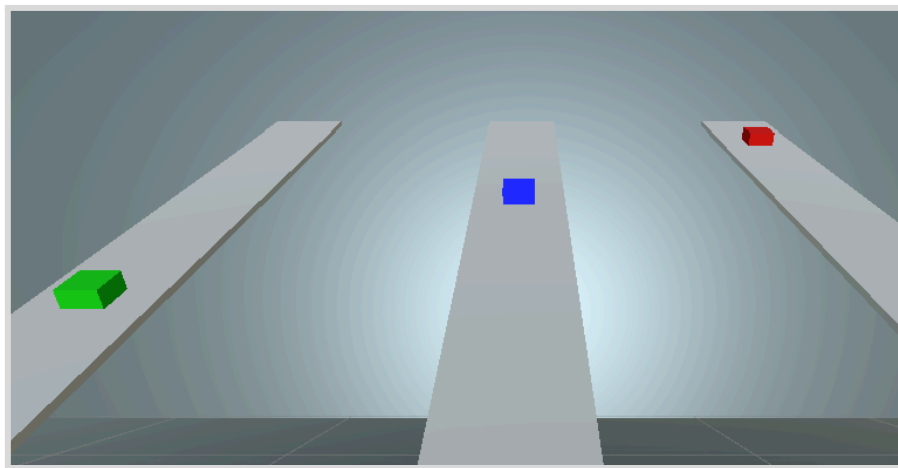


Figure 2. Unity Simulation of Low (green), Medium (blue), and High (red) Friction Conditions

This test compares how accurately Gazebo (ODE) and Unity (PhysX) physics engines simulate friction by having a box slide down a ramp with different friction conditions. The

goal of this test is to understand how each engine handles friction physics and how resulting simulations compare to theoretical expectations.

4.3.2.1 Experimental Setup

In this experimental scenario, a 1 kg box starts at the top of a long 30° inclined ramp. Gravity is the force that pulls it down the ramp, after reaching the bottom it then slides along the flat ground until friction stops it.

Box Specifications:

- Dimensions: 1m x 1m x 0.5 m (length x width x height)
- Mass: 1 kg
- Material: Rigid body (non-deformable), low/medium/high friction physics material
- Initial Orientation: Aligned with ramp surface, 30° from the horizontal

Environment Parameters:

- Ramp angle: 30° from the horizontal
- Ramp length: 30 m
- Gravity: -9.81 m/s²
- Data sampling rate: 50 Hz (0.02 s intervals)

Friction Conditions Tested:

- Low Friction: $\mu = 0.1$
- Medium Friction: $\mu = 0.3$
- High Friction: $\mu = 0.5$

These friction conditions were chosen to make sure the box always slid down the ramp instead of getting stuck or staying at rest.

4.3.2.2 Implementation

In Gazebo, the inclined-plane test is set up as a standalone SDF world that uses the ODE physics engine and a small fixed time step to get smooth motion. The world contains: a large flat ground plane, a 30 m long ramp tilted to 30°, and a 1 kg box that starts at the top of the

ramp. For each friction level, a separate world file is used and the friction parameters μ and μ_2 are set to the chosen coefficient on the ground, ramp, and box, so the contact behaviour is consistent across all surfaces. The box model is placed with its base aligned to the ramp surface so it immediately starts sliding once gravity is enabled.

An equivalent set up is also produced in Unity, using the PhysX engine and a C# script to log the data. The scene contains the same ramp and ground layout and an equivalent rigid body box. For each friction case, a different Physic Material is assigned with static and dynamic friction set to the same value and the friction combine mode set to “Minimum.” A recording script attached to the box starts recording the motion using the world coordinates and converts Unity’s Y-up coordinate frame to match Gazebo’s Z-up coordinate frame.

4.3.2.3 Data Collection

Both simulators recorded these values at 50 Hz:

- Time (s)
- Position (m)
- Velocity components (m/s)
- Overall speed (magnitude of velocity vector)

The simulations automatically stopped when the box velocity dropped below 0.001 m/s. Everything was saved to CSV files for analysis.

4.3.2.4 Theoretical Comparison

To compare the difference from expected behaviour between both simulators, a two phase theoretical model was considered. The first phase accounts for the box sliding down the ramp, where it is calculated to accelerate down: $a = g(\sin\theta - \mu\cos\theta)$.

The second phase begins with the box sliding on the flat ground after leaving the ramp, where it decelerates due to friction with: $a = -\mu g$

The box stops when its velocity hits zero. This theoretical model allows for comparison to theoretical physics. Although there are some difficulties in modeling the exact transition from the box sliding down the ramp to moving horizontally on the floor, this theoretical model

gives an idea of the relative behaviour and duration to help with directly comparing between the platforms.

4.3 Success Criteria for Test Implementation

Tests will be considered successfully completed if:

- The test runs without crashing or errors
- Data is logged to CSV format
- Quantitative metrics can be obtained to develop meaningful comparisons
- Results can be reproduced in repeated trials

5. Results

The following sections outline the results from both of the test scenarios used to compare and quantify some of the differences between the physics approach used by Unity and Gazebo.

5.1 Test 1: Joint-Space Trajectory Tracking

This section includes graphs and tables to illustrate the data and comparison of each platform from the first test. This test was focused on understanding how each platform can follow a desired trajectory.

5.1.1 UF850 Trajectory Tracking

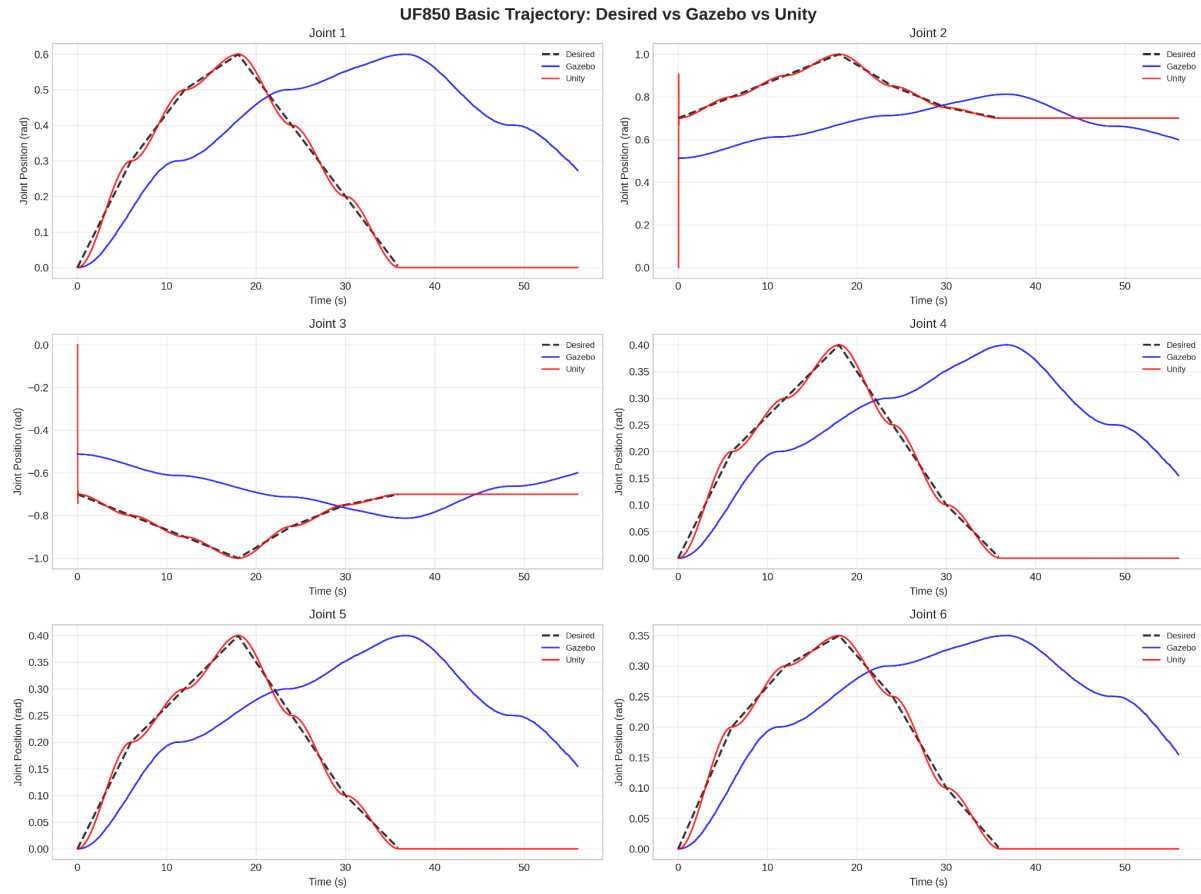


Figure 3. UF850 Basic Trajectory Tracking Data

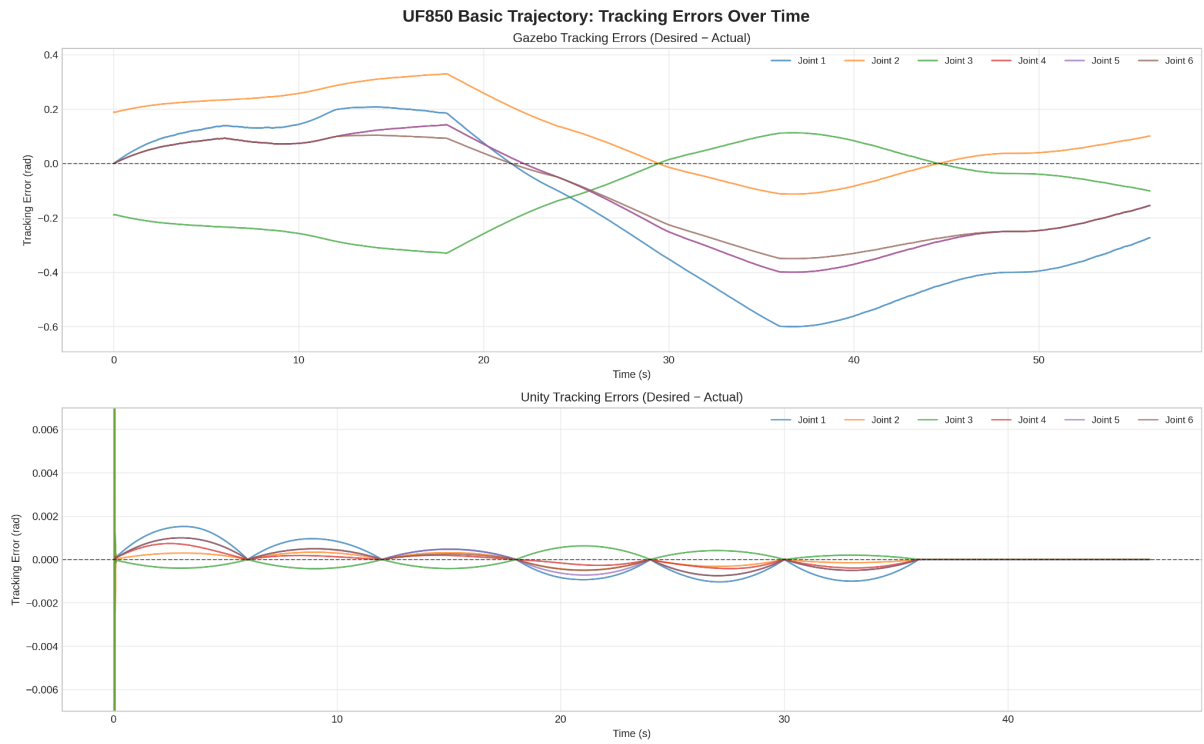


Figure 4. UF850 Basic Trajectory Tracking Error

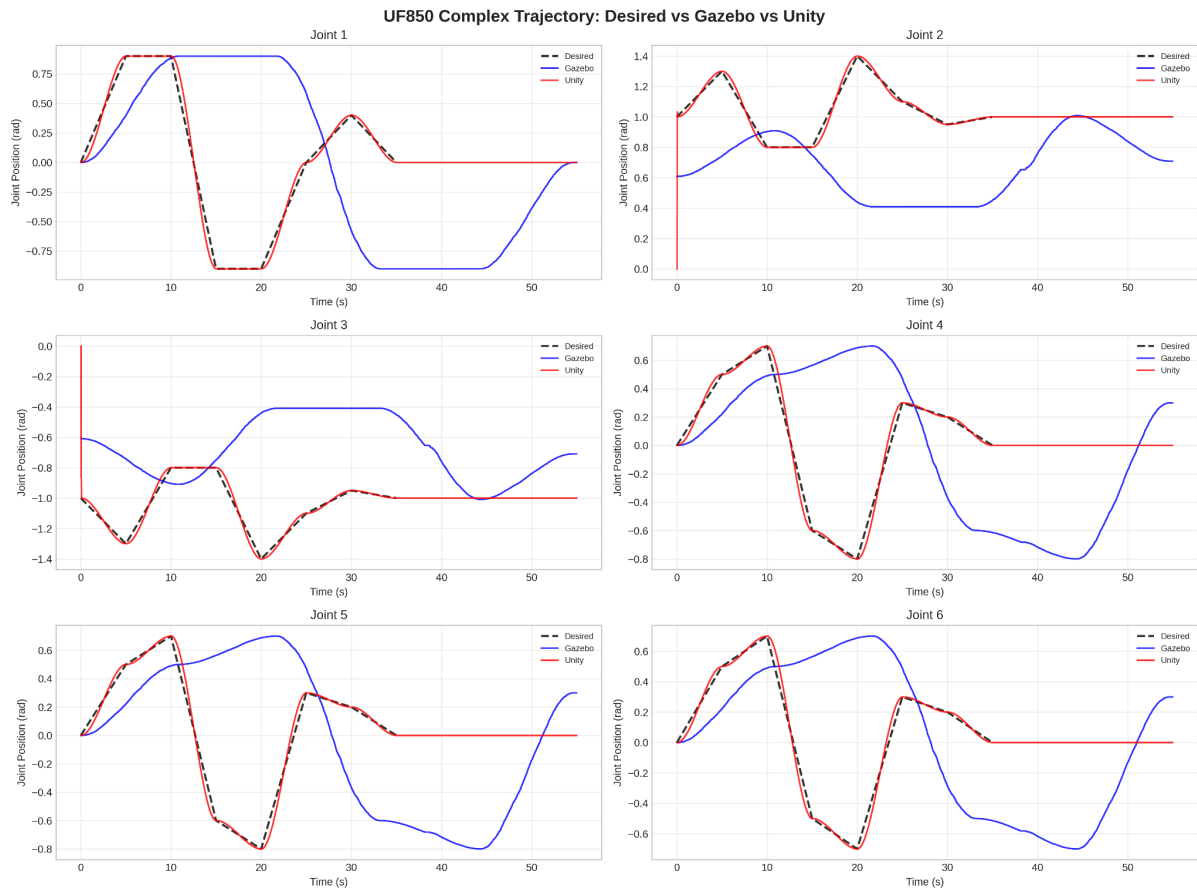


Figure 5. UF850 Complex Trajectory Tracking Data

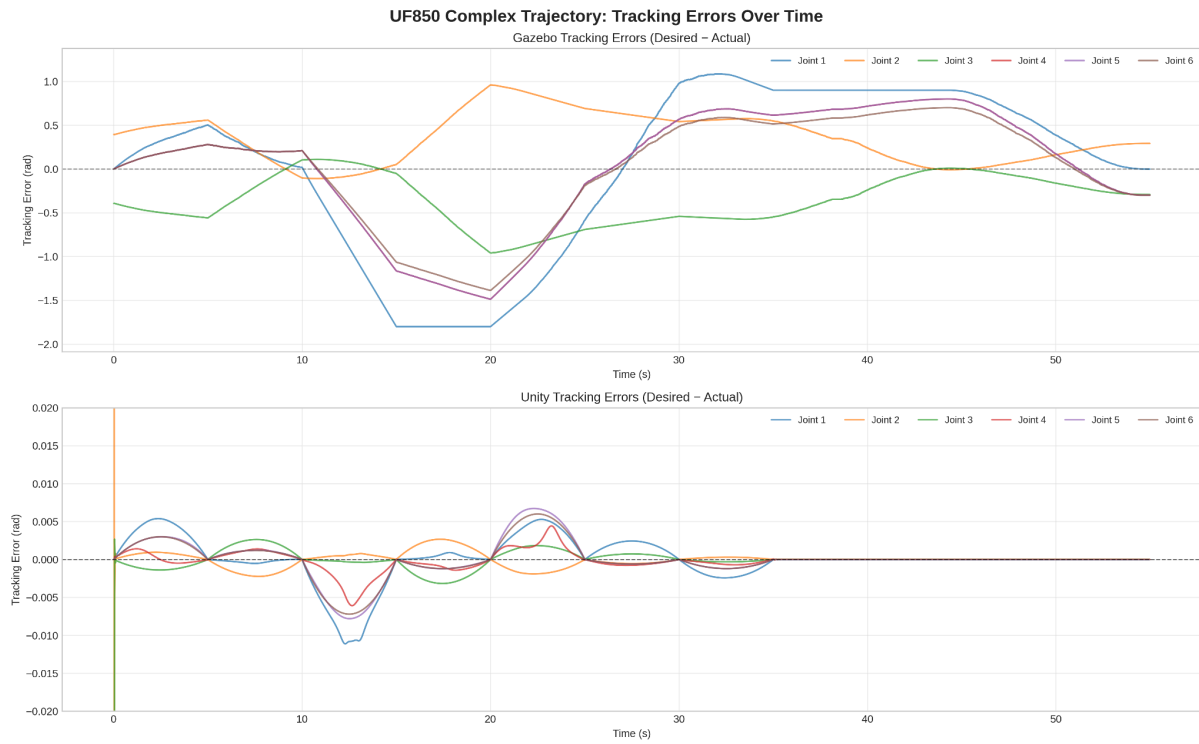


Figure 6. UF850 Complex Trajectory Tracking Error

5.1.2 xArm6 Trajectory Tracking

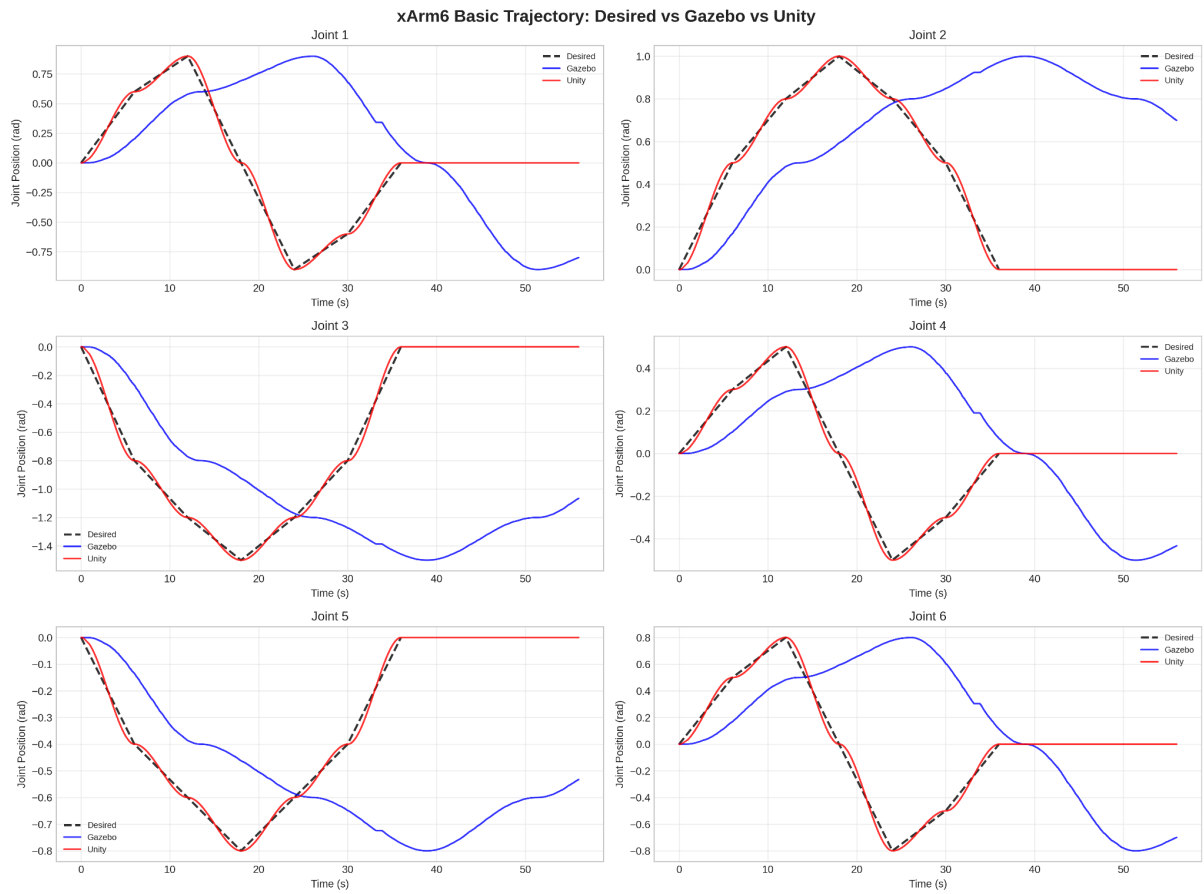


Figure 7. xArm6 Basic Trajectory Tracking Data

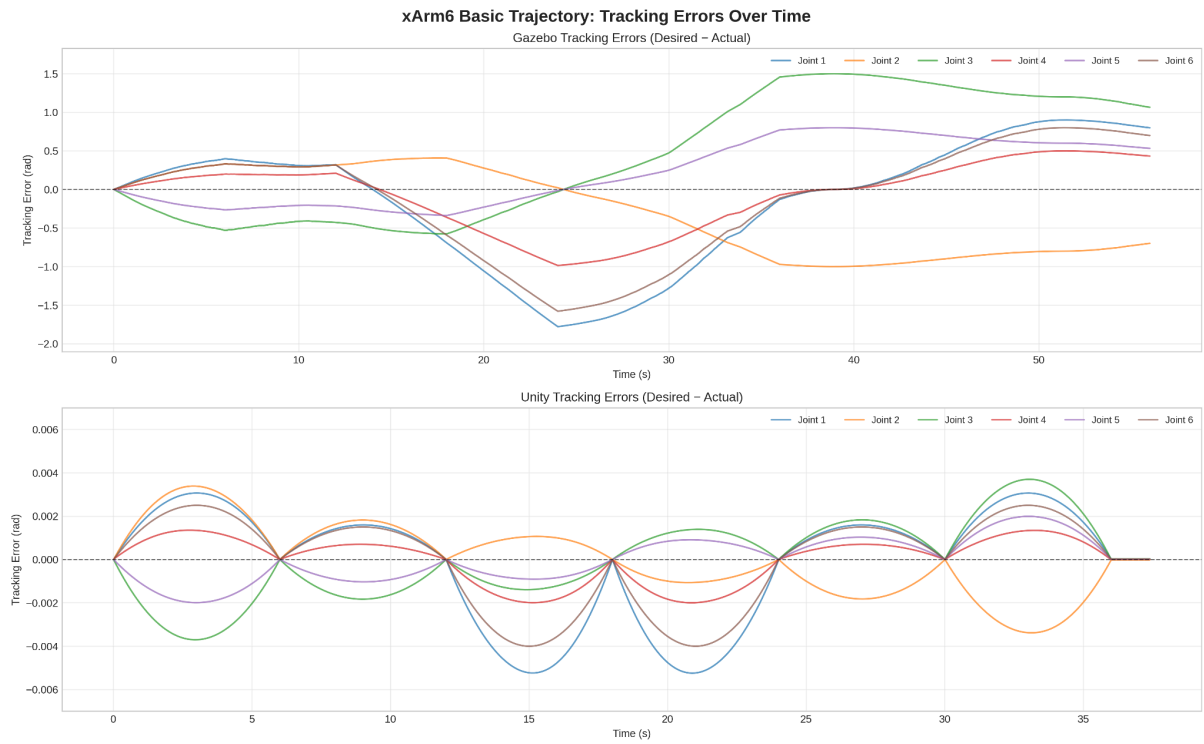


Figure 8. xArm6 Basic Trajectory Tracking Error

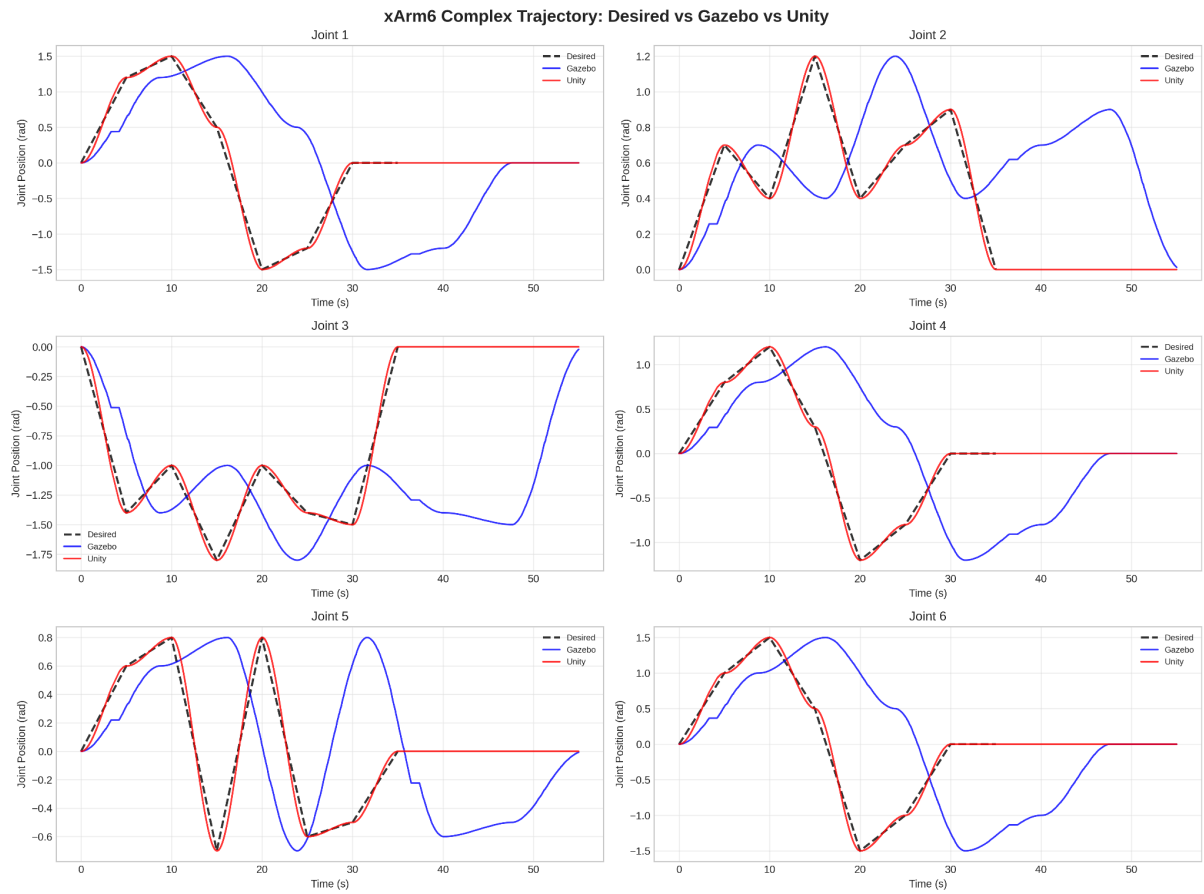


Figure 9. xArm6 Complex Trajectory Tracking Data

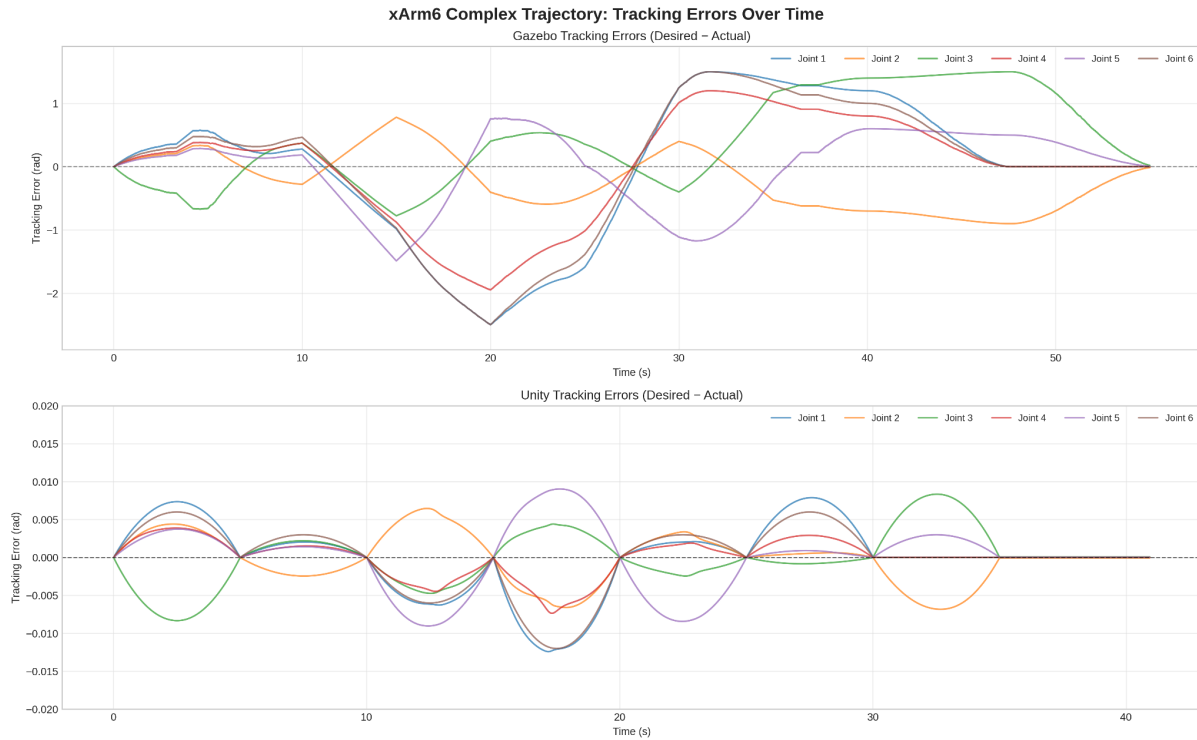


Figure 10. xArm6 Complex Trajectory Tracking Error

5.1.3 Summary of Trajectory Tracking Performance

Table 4. Summary of Trajectory Tracking Results for all Trials

Test (Robot & Trajectory)	Gazebo Mean Error (°)	Unity Mean Error (°)	Gazebo Max Error (°)	Unity Max Error (°)
UF850 Basic Trajectory	10.574	0.023	23.012	13.411**
UF850 Complex Trajectory	29.55	0.065	77.16	19.408**
xArm6 Basic Trajectory	31.534	0.08	72.968	0.194
xArm6 Complex Trajectory	36.755	0.149	103.313	0.534

****Note :** Unity max errors for the UF850 are outlier values when compared to the mean error. This could be a result of issues with the prescribed path specifically for the UF850 model.

5.2 Test 2: Friction and Sliding on an Inclined Plane

The following section outlines the results of the second test scenario using tables and graphs to illustrate how each platform handles friction and surface interactions.

5.2.1 Overview of Results

Table 5. Max Speed Data from Each Platform and Friction Condition

Friction Condition	Gazebo Max Speed (m/s)	Unity Max Speed (m/s)	Theoretical Max Speed (m/s)
Low ($\mu = 0.1$)	15.304	15.248	15.599
Medium ($\mu = 0.3$)	11.626	11.64	11.89
High ($\mu = 0.5$)	6.137	6.151	6.279

Table 6. Distance Data from Each Platform and Friction Condition

Friction Condition	Gazebo Distance (m)	Unity Distance (m)	Theoretical Distance (m)
Low ($\mu = 0.1$)	107.44	91.686	154.019
Medium ($\mu = 0.3$)	36.264	42.457	54.019
High ($\mu = 0.5$)	26.366	30.142	34.019

Table 7. Duration Data from Each Platform and Friction Condition

Friction Condition	Gazebo Duration (s)	Unity Duration (s)	Theoretical Duration (s)
Low ($\mu = 0.1$)	15.852	13.596	19.747
Medium ($\mu = 0.3$)	6.824	8.156	9.086
High ($\mu = 0.5$)	9.169	10.756	10.835

5.2.2 Position vs Time Analysis

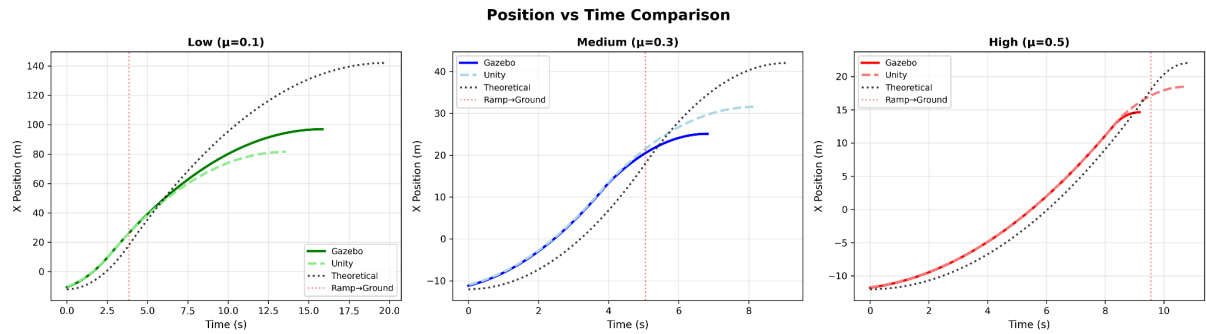


Figure 11. Position vs Time Comparison for Each Platform and Friction Condition

5.2.3 Speed vs Time Analysis

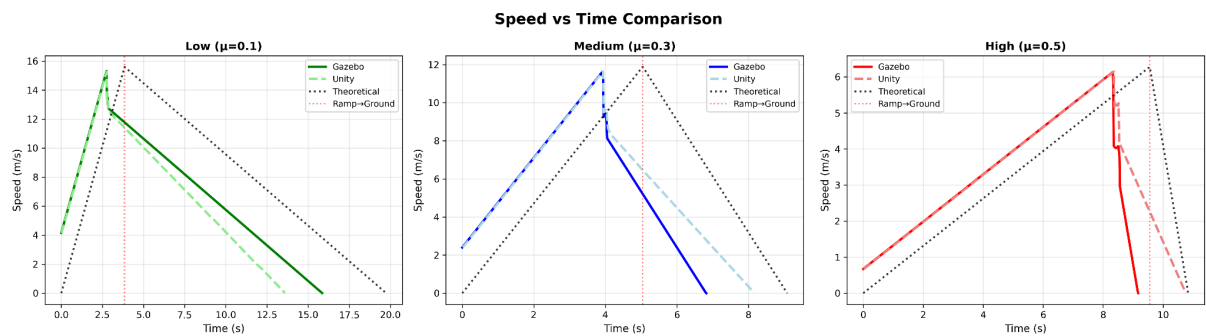


Figure 12. Speed vs Time Comparison for Each Platform and Friction Condition

5.2.4 Comparison of Performance Metrics

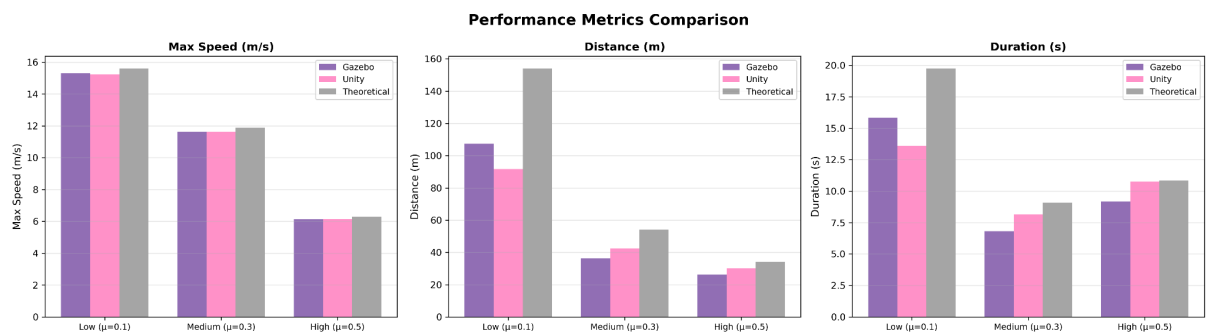


Figure 13. Comparison of Performance Metrics for Each Platform and Friction Condition

5.2.5 Error Analysis

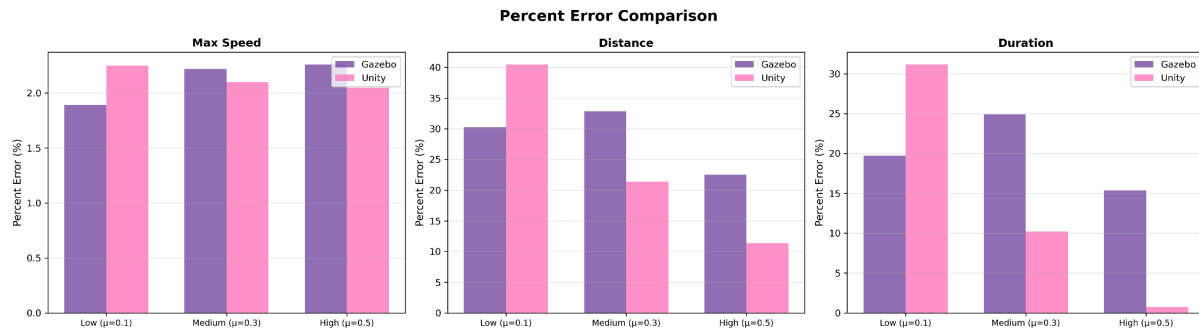


Figure 14. Percent Error Comparison for Each Platform and Friction Condition

Table 8. Summary of Error Analysis for Each Friction Condition and Platform

Friction Level	Platform	Max Speed Error (%)	Distance Error (%)	Duration Error (%)
Low ($\mu = 0.1$)	Gazebo	1.89	30.24	19.73
	Unity	2.25	40.47	31.15
Medium ($\mu = 0.3$)	Gazebo	2.22	32.87	24.89
	Unity	2.1	21.4	10.24
High ($\mu = 0.5$)	Gazebo	2.26	22.5	15.38
	Unity	2.05	11.4	0.73

6. Discussion

The following sections discuss the experimental results and the observed differences between simulators, explaining how these differences affect simulator selection, configuration, and the interpretation of simulation behaviours.

6.1 Test 1: Joint-Space Trajectory Tracking

6.1.1 Tracking Accuracy: Gazebo vs Unity

In all four trajectory tracking tests, Unity tracks the desired motion much more closely than gazebo. The mean absolute tracking errors stay below approximately 0.15° for each robot and trajectory, while Gazebo's tracking errors range from 10° - 37° depending on the test. The maximum errors show a much larger gap where Unity's largest deviation is 20° while Gazebo's is over 100° .

The graphs provide important insight as to where these large errors come from. In Gazebo, the joints consistently lag behind the desired motion and do not finish the path within the defined 36 seconds. The blue lines representing Gazebo joint motion always continue moving past the time set for the desired trajectory. It takes much longer for the joints to reach each desired position. Meanwhile, the Unity line in red closely follows the timing and motion of the desired path with errors magnitudes smaller than the errors seen in Gazebo.

Each simulator executes the same defined points with interpolation between points. In Gazebo, the trajectory is defined using a ROS2 *JointTrajectory* message that is sent to the joint trajectory controller once at the beginning. The joint trajectory controller then tries to follow the desired trajectory within its own internally defined limits for velocity and acceleration. With the UF850 and xArm6 configurations it seems these limits could be too conservative resulting in the joints not meeting their desired position in the schedule time. This could be the cause of the full Gazebo path taking much longer to complete than the time spacing between points in the described trajectory. In contrast, unity uses a direct control approach where the same points are input directly into the C# scripts to update each joint's *ArticulationBody* target angle at every timestep. This per step position control allows unity to stay on the desired timing and minimize tracking errors.

6.1.2 Effect of Trajectory Complexity

Changing from a relatively simpler trajectory to a more complex trajectory has the expected impact on tracking, especially visible in the Gazebo collected data. For both robots, the mean tracking error increases from a range of 10° - 32° in the simpler tests to 30° - 37° with the

more complicated trajectories. Unity also sees a small increase in error with more complex motion as expected.

The complex trajectories utilize larger joint ranges and more frequent changes in direction which requires higher accelerations and precise timing control. In Gazebo, the controller and model configuration are not able to keep up with increases in complexity of the path and it causes the joints to fall further from the expected trajectory. In comparison, Unity's control algorithm handles more complex joint-space commands with more precise timing and smaller deviations from the desired motion.

6.1.3 Implications for Choosing a Simulator

The results from this trajectory tracking experiment show that simulator choice is related to how much controller tuning is required before joint motion can accurately reflect a defined desired trajectory. Unity gives very accurate tracking with very little setup. Once the robot models and path points are input, the built-in articulation drives can allow the simulated joints to follow the path with only minor deviations in position and timing. In Gazebo, the same trajectories show how sensitive the motion is to the configuration of the controller and robot model parameters. Without improved tuning of the ROS2 joint trajectory controller, the robot joints are much slower than the defined motion which limits how well Gazebo can represent the intended trajectory. In practice, Gazebo requires much higher effort than Unity when it comes to initial configuration to produce reliable trajectory tracking results.

6.2 Test 2: Friction and Sliding on an Inclined Plane

6.2.1 Overview of Results

For all three friction conditions, Gazebo and Unity produced very similar max speeds that were close to the theoretical models. The maximum speed errors for both platforms stay below 2.5% for low, medium, and high friction tests, suggesting that the balance between gravity and friction on the ramp is handled consistently by both physics engines. The largest errors occur in the distance and duration data. Gazebo's distance errors range from approximately 22% to 33%, while Unity's distance errors range from about 11% to 21% for medium and high friction and increase to about 40% for low friction conditions. Duration

errors follow a similar pattern, with Gazebo showing a relatively consistent range of about 15% to 25%, and Unity ranging from about 0.7% to 10% for medium and high friction but over 30% for low friction. The largest discrepancies occur for Unity at low friction, whereas for medium and high friction conditions, Unity is closer to the expected values than Gazebo.

As the friction coefficient increases, the theoretical curves show shorter sliding distances and shorter durations as expected. Both Gazebo and Unity curves follow this trend, however, in all friction cases both simulators stop the box earlier than the theoretical model predicts, and Gazebo stops it before Unity except for in the low friction case.

At low friction, the numerical differences in each simulator have more time to accumulate. This explains why Unity produces results that are much closer to the theoretical expectations than Gazebo for medium and high friction conditions, but has much less accurate results for low friction. When $\mu = 0.1$, the box reaches a higher speed and spends more time sliding on the flat ground, so every time step where the simulator's numerical integrator adds a bit of damping or other small losses contributes to shortening the motion. Over several seconds, these small losses accumulate and Unity's box travels a much shorter distance than both Gazebo and the theoretical model. At medium and high friction, the physical friction force is stronger and the box stops much faster, so the same numerical effects only act for a short time and have less influence on the total motion. In those cases, Unity's results end up being closer to the theoretical distances and durations, while Gazebo tends to stop the box a bit earlier.

6.2.2 Implications for Choosing a Simulator

When choosing between Gazebo and Unity for this type of friction or contact scenario, the main implication of the results is that both simulators are acceptable choices if the main concern is matching peak speeds. The results for maximum speeds suggest that their handling of gravity, friction on the ramp, and basic integration of the equations of motion is consistent with the theoretical model.

The distance and duration data show that the choice becomes more important when stopping position and total sliding time are critical. Gazebo produced distance errors in the low- to mid-30% range and duration errors around 15–25% for all friction levels, which means it tends to stop the box earlier than theory but in a fairly consistent way that could be corrected for. Unity behaves differently, for medium and high friction it actually outperforms Gazebo,

with noticeably smaller distance and duration errors. However, at low friction its errors grow much larger because small numerical losses accumulate over the longer slide and cause the box to stop too soon. This suggests that for typical medium or high friction robotics tasks, Unity may provide slightly more accurate end behaviors, while for low-friction or long-sliding scenarios Gazebo's more consistent behavior may be more reliable. These observations demonstrate the importance of evaluating the performance of a simulator under task-relevant conditions so that results can be interpreted and any corrections and calibrations can be applied.

7. Future Work

7.1 Refining Experiment Design

Both experiments provide insight on differences between the platforms, however there are future steps that should be taken to get more interpretable and useful data. For the trajectory tracking experiment, the main limitation is the definition of the "basic" and "complex" trajectories. Since the robots used are quite complex, getting a more clean basic trajectory that only involves a single joint would make it easier to isolate trajectory tracking behaviour for different joints. Then in comparison the complex trajectory should involve multiple joints to see how tracking errors and controller limitations are highlighted clearly between both cases. A second limitation is the limited understanding and control of the internal parameters such as gains, filters, and update rates, as well as joint torque or velocity limits. For example, if a joint is past its torque limit it could appear to be bad at tracking even though it is just restricted by the simulator settings. Better understanding these parameters can help to separate modeling or control issues from implementation issues to get trajectory tracking data that is easier to interpret and compare.

For the friction test, one limitation was the ramp and horizontal phases being combined into a single test. This mixes inclined-plane and flat-ground friction and creates a transition region that is not accounted for properly in the simple theoretical model. Separating this into two experiments with one focusing on motion down the incline and one focusing on flat sliding with initial velocity, would make it easier to understand where errors are introduced and have clearer cleaner distance and duration measurements. Additionally, the current tests rely on

analytical models that make their own assumptions, so differences could reflect limits of the theory rather than of the simulators themselves.

7.2 Further Platform Comparison

Future steps for platform comparison could include refining the two current test scenarios and implementing them on Isaac Sim as originally intended. On top of expanding the platforms considered in these experiments, the same type of analysis can be repeated on more complex tasks such as precise pick and place tasks or sensor-integrated tasks. These types of scenarios could help not just evaluate the underlying physics accuracy and how well each simulator handles realistic sensing and control tasks.

7.3 Real World Validation

Real-world validation is an important next step to see how well these simulation results transfer to actual hardware. A natural extension of this work would be to repeat at least one of the trajectory tracking experiments on the physical UF850 or xArm6 and record the same commanded and measured joint positions. Comparing those data directly to the simulated trajectories would make it possible to tune friction coefficients, damping values, and controller gains so that the simulations better match the behaviour of the real robot.

Similarly, the friction and sliding experiment could be recreated with a physical ramp and flat surface using a box of known mass and surface properties. Measuring real sliding distances and durations for the same friction settings would show whether the ideal theoretical model, Gazebo, or Unity is closer to what actually happens. If the real data is different from all three that would give insight into how to improve the model to include effects and losses that are not currently considered to create a more realistic reference.

8. References

1. A. M. Law and W. D. Kelton, *Simulation Modeling and Analysis*, 5th ed. New York, NY, USA: McGraw-Hill Education, 2015.
2. F. P. Audonnet, A. Hamilton, and G. Aragon-Camarasa, "A systematic comparison of simulation software for robotic arm manipulation using ROS2," *arXiv preprint arXiv:2204.06433*, 2022.
3. S. M. Kargar, B. Yordanov, C. Harvey, and A. Asadipour, "Emerging Trends in Realistic Robotic Simulations: A Comprehensive Systematic Literature Review," *IEEE Access*, vol. 12, pp. 191264–191287, May 2024, doi: 10.1109/ACCESS.2024.3404881.
4. Q. Le Lidec, W. Jallet, L. Montaut, I. Laptev, C. Schmid, and J. Carpentier, "Contact Models in Robotics: a Comparative Analysis," *arXiv preprint arXiv:2304.06372*, 2023.
5. S. Zickler, "Physics-Based Simulation for Real-Time Robotic Manipulation," Ph.D. dissertation, Carnegie Mellon Univ., Pittsburgh, PA, USA, 2010.
6. K. Liu and D. Negrut, "The Role of Physics-Based Simulators in Robotics," *Annual Rev. Control, Robotics, and Autonomous Systems*, vol. 4, pp. 35-58, May 2021, doi: 10.1146/annurev-control-072220-093055.
7. R. K. Thaker, "Physics-Based Simulation for Robotics: Simulating real-world environments for training and validation," *Int. J. Leading Research Publication*, vol. 3, no. 6, pp. 1–6, Jun. 2022.
8. J. Yoon, B. Son, and D. Lee, "Comparative Study of Physics Engines for Robot Simulation with Mechanical Interaction," *Applied Sciences*, vol. 13, no. 2, art. 680, Jan. 2023, doi: 10.3390/app13020680.
9. J. Collins, S. Chand, A. Vanderkop, and D. Howard, "A review of physics simulators for robotic applications," *IEEE Access*, vol. 9, pp. 51416–51434, Mar. 2021, doi: 10.1109/ACCESS.2021.3068769.
10. M. Kaup, C. Wolff, H. Hwang, J. Mayer, and E. Bruni, "A Review of Nine Physics Engines for Reinforcement Learning Research," *arXiv preprint arXiv:2407.08590*, 2024.
11. M. Singh, J. Kapukotuwa, E. L. S. Gouveia, E. Fuenmayor, Y. Qiao, N. Murry, and D. Devine, "Unity and ROS as a digital and communication layer for digital twin

application: Case study of robotic arm in a smart manufacturing cell,” *Sensors*, vol. 24, no. 17, p. 5680, 2024, doi: 10.3390/s24175680.

12. RidgeRun, “NVIDIA Isaac Sim, Omniverse, and COSMOS: The robotics AI simulation ecosystem explained,” *RidgeRun AI Blog*, Feb. 13, 2024. Available: <https://www.ridgerun.ai/post/nvidia-isaac-sim-omniverse-and-cosmos-the-robotics-ai-simulation-ecosystem-explained>
13. Z. Zhou, J. Song, X. Xie, Z. Shu, L. Ma, D. Liu, J. Yin, and S. See, “Towards Building AI-CPS with NVIDIA Isaac Sim: An Industrial Benchmark and Case Study for Robotics Manipulation,” arXiv preprint arXiv:2308.00055, 2023.
14. F. Muratore, F. Ramos, G. Turk, W. Yu, M. Gienger, and J. Peters, “Robot learning from randomized simulations: A review,” *arXiv preprint arXiv:2111.00956*, 2022.
15. E. Aljalbout et al., “The Reality Gap in Robotics: Challenges, Solutions, and Best Practices,” arXiv preprint arXiv:2510.20808, 2025.
16. UFACTORY, UFactory 850 User Manual, Version 2.1.0, 2023. Available: <https://www.ufactory.cc/wp-content/uploads/2023/07/UFactory-850-User-Manual-V2.1.0.pdf>
17. UFACTORY, xArm User Manual, Version 2.0.0, 2023. Available: <https://www.ufactory.cc/wp-content/uploads/2023/05/xArm-User-Manual-V2.0.0.pdf>